A One-State Recursor for iPRA

Recovering arbitrary initialization through a transient seed

Research prepared for Vladimir Reshetnikov

10 November 2026

Abstract

The shared-continuation draft leaves a specific two-constructor arithmetic candidate unresolved: a scalar recursor with successor seed, whose update receives the previous state but not the counter. We close that gap. A transient state supplies selected zero-start iterations before arbitrary initialization is available. These produce a comparator; a reset cycle then gives a finite predecessor term by literal-equation induction. The initializer compiler consequently recovers ordinary iteration.

A second bootstrap makes this result available directly in the small native logic. A finite decoder cancels successor on Boolean values before a general predecessor exists. It supplies restricted implication and contextual Boolean case analysis with equation conclusions, exactly what the arithmetic proof needs. Only afterward is the unshifted selector used to reconstruct full intuitionistic first-order logic. The resulting $0, R$ presentation, with one update slot and two computation schemas, is whole-domain bi-interpretable and equiconsistent with full iPRA. Both numerical composites are the identity, and the native Q/D formula composites are proved.

A complete direct SR checker occupies 681 normalized bytes, below the 685-byte minimum in the supplied current corpus. Its 284 tokens and 322 AST nodes do not beat the respective corpus minima 281 and 315. An auxiliary initialized-I predecessor-packet checker has profile 280/314/684, strictly below all three incumbent coordinates in one complete program. Further auxiliary profiles give the separate coordinate envelope 280/311/679; no one program attains that whole new envelope. The principal result is the arithmetic and logical bootstrap, not those small implementation savings.

A separate obstruction theorem proves that tying a recursor's seed to its own update forces every numerical term to have a single bounding input coordinate. That candidate cannot represent addition, a general selector, or injective pairing by terms. Explicit rules, internal proofs, source provenance, executable certificates, numerical diagnostics, and reproduction scripts are included. Absolute minimality, literature-wide priority, and proof-assistant verification are not claimed.

Contents

1 What improves, and what the comparison establishes **4**
1.1 The new arithmetic result . . . 4
1.2 Closing the logical bootstrap, and the remaining boundary . . . 5

2 The complete ordinary axiomatization **5**
2.1 Every nonlogical axiom and induction rule . . . 5
2.2 Every logical axiom and rule . . . 6

3 A comparator from a successor-seeded scalar recursor **6**
3.1 A positive selector and two finite cycles . . . 7
3.2 The transient-seed construction . . . 7
3.3 A factor map with small affine constants . . . 9
3.4 Removing affine factors gives comparison . . . 9

4 Removing the independent successor constructor with one update slot **10**
4.1 Exact source theory and the comparator prerequisite . . . 10
4.2 Shifted addition and elementary comparison facts . . . 11
4.3 A reset cycle with an accessible transient seed . . . 12
4.4 A conditional equation induction, with its literal invariant . . . 12
4.5 Two ordinary equation inductions give the predecessor . . . 13
4.6 Ordinary initialization and both interpretation composites . . . 13

5 Completion to full iPRA without a standard-model shortcut **14**
5.1 A characteristic of actual equality . . . 14
5.2 Every quantifier-free induction instance . . . 15
5.3 Pairing and the general recursion compiler . . . 15
5.4 Both return interpretations . . . 16

6 The native checker comparison, with its exact boundary **17**
6.1 Its entire primitive inventory . . . 17
6.2 Why this is a full intuitionistic presentation . . . 17
6.3 Codec and scope . . . 18
6.4 What changed in the measured programs . . . 19
6.5 An optional predecessor-specified computation interface . . . 19
6.6 Evidence and its limits . . . 21

7 The native SR bootstrap without an unshifted selector **21**
7.1 Primitive setting and the two immediate derived rules . . . 21
7.2 A finite successor decoder, before comparison . . . 22

7.3 Restricted implication and genuine contextual Boolean cases . . . 22
7.4 The modulo-three cycle without a range disjunction . . . 23
7.5 Transferring the arithmetic proof, step by step . . . 23
7.6 Closing the full logical and arithmetic equivalence . . . 24

8 A bounded-coordinate obstruction for a tied-seed recursor 24
8.1 The candidate and its standard interpretation . . . 25
8.2 A property of every numerical term . . . 25
8.3 What the obstruction rules out . . . 27
8.4 Exact logical scope . . . 27

9 Why deleting atomic case introduction is not an equivalence 28

10 Further research questions 28

A The complete measured programs 30

B Reproduction and attribution 34

1 What improves, and what the comparison establishes

The request has two distinct demanding parts: a simpler arithmetic/logical presentation genuinely equivalent to intuitionistic PRA, and a complete proof checker below the best supplied implementations. We give a new arithmetic basis, a noncircular bootstrap of the native logical interface, and a direct complete SR checker below the incumbent normalized-byte minimum. We do not identify an attained source record with a proof of absolute optimality.

The comparison is pinned to the repository commit in [1]. All nine direct ZIP members of `docs/drafts` were read. The directory contains atomic transport, elimination assumptions, endpoint certificates, generated assumptions, guarded elimination, named specialization, scalar countdown, shared continuation, and successor recursion. Their historical comparison tables refer to older snapshots, so they are not themselves the current benchmark. The current best complete-source coordinates are

281 tokens, 315 AST nodes, 685 bytes.

The generated-assumptions main program attains 281/318/685, and its AST variant attains 283/315/710. These are attained upper bounds on the cost of a checker, not mathematical lower bounds on every possible axiomatization.

Complete program or coordinate envelope	Tokens	AST	Bytes
Current coordinate envelope	281	315	685
Current generated-assumptions main	281	318	685
Current generated-assumptions AST	283	315	710
New byte profile, same accepted language	282	320	679
New iterable profile, explicit additional aliases	280	311	705
Direct native SR byte profile	284	322	681
New I predecessor-packet profile	280	314	684
New coordinate envelope (different programs)	280	311	679

All imports, traversal, scope checks, proof dispatch, JSON input, and ACCEPT/REJECT output are charged under the unchanged corpus normalizer. The new coordinate reductions are respectively 0.36%, 1.27%, and 0.88%. No single listed program attains the whole new envelope. Nevertheless the predecessor-packet I profile is a single complete program strictly below all three incumbent coordinate minima: 280/314/684. The direct SR checker costs 681 normalized bytes, four fewer than the incumbent byte record, while using more tokens and AST nodes. Its successor node is a charged abbreviation, and its complete input and checking code is included. The small source changes should not be described as the principal mathematical result.

1.1 The new arithmetic result

The successor-recursion draft [3] uses $R(n, a; u, v.h)$, with separate old-counter and old-state inputs. The shared-continuation draft [2] asks whether the one-state $R(n, a; v.h)$ can also dispense with a primitive successor. It isolates the missing theorem $P(Sa) = a$ and correctly warns that the old scalar predecessor proof already assumes arbitrary initialization.

We supply that missing theorem. The new construction first creates selected zero-start trajectories using transient seeds, then creates a Boolean comparator, and finally builds a parameterized reset cycle. Three explicit equation invariants yield predecessor. The initializer compiler

$$\widehat{I}(n, a; u.h) = P\big(R(n, a; v.S(h[Pv/u]))\big)$$

then satisfies both ordinary scalar recursion equations by computation. This is a whole-domain argument: the intermediate facts are internal equations, and every induction holds all passive parameters fixed.

The numerical grammar therefore has two primitive constructors, $0, R$; its update binds one variable; and it uses two computation schemas. The ordinary logical rules, coverage, separation, and equation induction are all stated explicitly below. This is not a finite ordinary first-order signature with a universal evaluator: each body is fixed earlier finite syntax, or equivalently names a function in a staged signature.

1.2 Closing the logical bootstrap, and the remaining boundary

The native Choice-DI logic in the supplied small programs reconstructs ordinary logic using an unshifted selector. Applying that reconstruction before proving predecessor would be circular. Section 7 closes this additional gap: successor is already cancellable on Boolean values by a finite decoder. This yields a restricted native implication and contextual Boolean case elimination with equation conclusions. Those are exactly the case analyses used in the comparator and reset-cycle proofs. Thus predecessor is derived in native SR itself; the unshifted selector and full ordinary logical interface are reconstructed afterward.

The direct program `code/checker_sr_bytes.py` checks the new SR computation schemas, with a zero-counter result Sa rather than a. Its two-byte change to the arithmetic supply is charged in full. The other profiles remain initialized-I checkers and are labelled as such. Both native presentations are bi-interpretable with iPRA, including native-formula return translations and numerical identity composites.

A globally simplest axiomatization and a lower bound applicable to arbitrary languages and domains are not claimed. The optional I predecessor-packet profile does beat all three attained incumbent minima in one complete program; the native SR program separately beats their byte minimum. The closed result is a genuine arithmetic and logical reduction, accompanied by a direct implementation below the supplied byte frontier.

2 The complete ordinary axiomatization

Use one numerical sort and finite terms

$$t ::= x \mid 0 \mid R(n, a; u.h), \qquad Sa := R(0, a; u.0).$$

Only h binds u. Bound names are renamed before capture-free substitution. Passive parameters may be arbitrary numerical terms; the description of each update is fixed finite earlier syntax. Formulas are equations, genuine falsity $\bot$, and their closure under $\wedge, \vee, \to, \forall, \exists$. Negation abbreviates implication to $\bot$. We use the usual full first-order iPRA, for which atomic induction is a standard presentation [6]; the equivalence to quantifier-free induction is proved in Section 5.

2.1 Every nonlogical axiom and induction rule

The computation schemas are exactly

$$R(0, a; u.h) = Sa, \tag{SR0}$$
$$R(Sn, a; u.h) = h[R(n, a; u.h)/u]. \tag{SRS}$$

There are two additional boundary sentences:

$$\forall x\,(x = 0 \vee \exists y\, x = Sy), \qquad (0 = S0) \to \bot. \tag{Cov,Sep}$$

The only primitive induction is the contextual rule for a literal equation $E(n, \vec{p})$:

$$\frac{\Gamma \vdash E(0, \vec{p}) \qquad \Gamma, E(z, \vec{p}) \vdash E(Sz, \vec{p})}{\Gamma \vdash E(t, \vec{p})}. \tag{EI}$$

Here z is absent from $\Gamma, t, \vec{p}$. The context may contain quantified formulas. Induction on an implication, a disjunction, or an arbitrary quantified formula is not primitive. There is no additional successor-injectivity axiom, predecessor axiom, selector axiom, or initializer axiom.

2.2 Every logical axiom and rule

Contexts are finite reusable lists. The assumption, equality, and falsity rules are

$$\frac{}{\Gamma \vdash A}\ (A \in \Gamma), \qquad \frac{}{\Gamma \vdash t = t}, \qquad \frac{\Gamma \vdash s = t \quad \Gamma \vdash F(s)}{\Gamma \vdash F(t)}, \qquad \frac{\Gamma \vdash \bot}{\Gamma \vdash A}.$$

The equality motive F is any fixed finite formula. The propositional rules are precisely

$$\frac{\Gamma, A \vdash B}{\Gamma \vdash A \to B}, \qquad \frac{\Gamma \vdash A \to B \quad \Gamma \vdash A}{\Gamma \vdash B},$$

$$\frac{\Gamma \vdash A \quad \Gamma \vdash B}{\Gamma \vdash A \wedge B}, \qquad \frac{\Gamma \vdash A_0 \wedge A_1}{\Gamma \vdash A_i}\ (i = 0, 1),$$

$$\frac{\Gamma \vdash A_i}{\Gamma \vdash A_0 \vee A_1}\ (i = 0, 1), \qquad \frac{\Gamma \vdash A \vee B \quad \Gamma, A \vdash C \quad \Gamma, B \vdash C}{\Gamma \vdash C}.$$

The quantifier rules are

$$\frac{\Gamma \vdash A(z)}{\Gamma \vdash \forall x\, A(x)}, \qquad \frac{\Gamma \vdash \forall x\, A(x)}{\Gamma \vdash A(t)},$$

$$\frac{\Gamma \vdash A(t)}{\Gamma \vdash \exists x\, A(x)}, \qquad \frac{\Gamma \vdash \exists x\, A(x) \quad \Gamma, A(z) \vdash C}{\Gamma \vdash C}.$$

In universal introduction z is fresh for Γ and the passive parameters; in existential elimination it is additionally absent from C. Displayed local hypotheses are discharged. Numerical substitution, renaming, weakening, exchange, contraction, and ordinary assumption cut are admissible by induction on finite derivations. No excluded middle, double-negation elimination, or other classical rule is included.

This explicitly defined theory is denoted T_{SR}. The comparator and predecessor constructions below use this ordinary logical interface. That choice is part of the cost and dependency statement, not an unlisted logical oracle.

3 A comparator from a successor-seeded scalar recursor

This section works in ordinary intuitionistic first-order logic with actual equality. The numerical language has 0 and the scalar recursor

$$R(n, a; v.h), \qquad R(0, a; v.h) = Sa, \qquad R(Sn, a; v.h) = h[R(n, a; v.h)/v],$$

where Sa abbreviates $R(0, a; v.0)$. Only v is bound in h; every other parameter remains fixed during a recursion. Besides the computation equations we assume zero/successor coverage, $(0 = 1) \to \bot$ with $1 = S0$, and contextual induction on a single literal equation. Every induction below has such an equation as its invariant. Case analysis on a previously proved finite range is an intuitionistic logical elimination, not an induction on that range formula. We do not assume predecessor, successor injectivity, ordinary initialized scalar iteration, addition, pairing, or an equality decision function.

For each fixed update $f(v)$, the notation $R(n, a; f)$ means its fixed finite term instance. Function notation below is abbreviation, never quantification over numerical functions. $S^k a$ uses a fixed standard number k of successor signs.

3.1 A positive selector and two finite cycles

Define

$$T(n, a, b) = R(n, a; v.b), \qquad Z(n) = T(n, 0, 0).$$

Their computation equations are

$$T(0, a, b) = Sa, \quad T(Sn, a, b) = b, \qquad Z(0) = 1, \quad Z(Sn) = 0.$$

Coverage therefore proves that Z takes only the values $0, 1$. Put

$$N(n) = Z(R(n, 0; Z)), \qquad j(u) = T(u, 0, T(Z(N(u)), 1, 0)), \qquad M(n) = j(j(R(n, 0; j))).$$

The range of $R(n, 0; Z)$ is contained in $\{0, 1\}$: its zero value is 1, and each successor value is a value of Z. Hence N is Boolean. Direct computation gives

$$N(0) = 0, \qquad N(Sn) = Z(N(n)), \qquad N(S^2 n) = N(n), \qquad N(S^3 n) = Z(N(n)). \tag{N}$$

The last two identities use only the Boolean range of N.

Coverage on u and the Boolean range of $N(u)$ show that $j(u)$ is always one of $0, 1, 2$. Finite computations give

$$j(0) = 1, \qquad j(1) = 2, \qquad j(2) = 0.$$

Consequently the range of M is contained in $\{0, 1, 2\}$ and

$$M(0) = 0, \quad M(Sn) = j(M(n)), \quad M(S^3 n) = M(n), \qquad M(1) = 1, \quad M(2) = 2. \tag{M}$$

Here $M(Sn) = j(M(n))$ follows by expanding one recursor step; the three-period identity follows by three such steps and the finite cycle. No induction asserting a disjunctive range has been used.

3.2 The transient-seed construction

For a fixed map f and a fixed seed a, the composite

$$J_f(n) = f(R(n, a; f))$$

satisfies $J_f(0) = f(Sa)$ and $J_f(Sn) = f(J_f(n))$. Thus a state Sa outside an intended orbit can be used once to enter that orbit. The following constructions exploit a residue absent from the subsequent orbit; they do not presuppose a general initialized iterator.

Define

$$f_2(u) = T(N(u), Su, 0), \qquad d_2(n) = f_2(R(n, 0; f_2)),$$
$$f_3(u) = T(M(u), S^2 u, 0), \qquad d_3(n) = f_3(R(n, 0; f_3)).$$

Since $f_2(1) = f_3(1) = 0$, both $d_2(0)$ and $d_3(0)$ are zero. Literal-equation induction proves respectively

$$N(d_2(n)) = 0, \qquad M(d_3(n)) = 0.$$

For the first step, the induction equation turns $f_2(d_2(n))$ into $S^2 d_2(n)$, whose N value is still zero by (N). The second proof uses $f_3(d_3(n)) = S^3 d_3(n)$ and (M) in the same way. We obtain the entire recursion equations

$$d_2(0) = 0, \quad d_2(Sn) = S^2 d_2(n), \qquad d_3(0) = 0, \quad d_3(Sn) = S^3 d_3(n). \tag{Scale}$$

These justify the intended doubling and tripling roles internally, without using their standard-model meanings as premises. A separate equation induction, using (N) and (Scale), proves

$$N(d_3(n)) = N(n). \tag{Parity-scale}$$

Indeed the successor step applies Z to both sides of the current induction equation.

Now put

$$f(u) = T\big(M(u), u, T(Z(N(M(u))), Su, 0)\big), \qquad G(n) = f(R(n, 1; f)).$$

On the three possible residues of $M(u)$, finite computations yield

$M(u)$	0	1	2
$f(u)$	Su	S^2u	0.

(Ramp-map)

In particular $f(2) = 0$. The seed $S1 = 2$ is therefore transient, and

$$G(0) = 0, \qquad G(Sn) = f(G(n)).$$

Equation induction proves the central invariant

$$M(G(n)) = N(n). \tag{Ramp-invariant}$$

At the successor step retain this one induction equation and split the Boolean $N(n)$. If it is zero, (Ramp-map) gives $G(Sn) = SG(n)$, and the new M value is $j(0) = 1 = N(Sn)$. If it is one, the new state is $S^2G(n)$, and its M value is $j^2(1) = 0 = N(Sn)$. Thus

$$G(Sn) = \begin{cases} SG(n) & N(n) = 0, \\ S^2G(n) & N(n) = 1, \end{cases} \qquad G(S^2n) = S^3G(n). \tag{Ramp-step}$$

One more step gives $G(S^3n) = S^4G(n)$ in the first Boolean case and $G(S^3n) = S^5G(n)$ in the second. Equation induction on n, using (Scale) and (Ramp-step), gives

$$G(d_2(n)) = d_3(n). \tag{Ramp-double}$$

For the other scaling identity define the conditional increment

$$\Delta(e, a) = T(Z(e), a, a).$$

It satisfies $\Delta(0, a) = a$ and $\Delta(1, a) = Sa$. We claim

$$G(d_3(n)) = \Delta\big(N(n), d_3(G(n))\big). \tag{Ramp-triple}$$

This is again one literal-equation induction on n. The base is zero. For the step write $b = d_3(G(n))$ and use (Parity-scale). If $N(n) = 0$, the induction equation says $G(d_3(n)) = b$. The new left side is $G(S^3 d_3(n)) = S^4 b$. The new right side is $\Delta(1, d_3(SG(n))) = S(S^3 b) = S^4 b$. If $N(n) = 1$, the induction equation says $G(d_3(n)) = Sb$. The new left side is $S^5(Sb) = S^6 b$, and the new right side is $\Delta(0, d_3(S^2G(n))) = S^6 b$. This proves the identity at every argument, including nonstandard arguments in a source model.

3.3 A factor map with small affine constants

Define

$$F_2(u) = Sd_2(u), \qquad F_3(u) = S^2 d_3(u), \qquad H(u) = T(Z(N(u)), G(u), u).$$

The preceding equations give

$$N(F_2(u)) = 1, \qquad N(F_3(u)) = N(u), \qquad H(u) = \begin{cases} u & N(u) = 0, \\ SG(u) & N(u) = 1. \end{cases} \tag{Factor-cases}$$

We now prove

$$H(F_2(u)) = F_3(u), \qquad H(F_3(u)) = F_3(H(u)). \tag{Factor}$$

For the first, $N(d_2(u)) = 0$ and (Ramp-double) give

$$H(F_2(u)) = SG(Sd_2(u)) = S^2 G(d_2(u)) = S^2 d_3(u).$$

For the second, split the Boolean $N(u)$. At zero both sides are $F_3(u)$. At one, (Ramp-step) and (Ramp-triple) give

$$H(F_3(u)) = SG(S^2 d_3(u)) = S^4 G(d_3(u)) = S^5 d_3(G(u)),$$

whereas $F_3(H(u)) = S^2 d_3(SG(u)) = S^5 d_3(G(u))$ by (Scale). Notice also that $H(2) = 2$, since $N(2) = 0$.

3.4 Removing affine factors gives comparison

Put

$$E(x) = R(x, 0; v.S^2 d_2(v)), \qquad Q(a) = S^4 d_3(a).$$

Their useful equations are

$$E(0) = 1, \qquad SE(Sx) = F_2(SE(x)), \qquad SQ(a) = F_3(Sa). \tag{Seeds}$$

The middle equation follows from $d_2(Sa) = S^2 d_2(a)$; the last from $d_3(Sa) = S^3 d_3(a)$.

For a fixed update h, if $h(Sa) = Sb$, equation induction on y proves

$$R(Sy, a; h) = R(y, b; h). \tag{Rotation}$$

Its base is the assumed equation, and its step applies h to the current induction equation. All initializer and update parameters stay fixed. In particular, (Factor) and (Seeds) supply

$$H(SE(Sx)) = F_3(SE(x)) = SQ(E(x)),$$

so (Rotation) gives

$$R(Sy, E(Sx); H) = R(y, Q(E(x)); H). \tag{Seed-rotation}$$

A further equation induction on y, with a fixed, proves

$$R(y, Q(a); H) = F_3(R(y, a; H)). \tag{Iterator-factor}$$

Its base is the last equation of (Seeds). Its step is precisely the second equation of (Factor), applied after the induction equation.

Finally define the finite scalar term

$$L(x, y) = Z(N(R(y, E(x); H))).$$

Its Boolean range is immediate from that of N and Z. The complete comparison recursion specification is internally provable:

$$\boxed{L(0,y)=1, \qquad L(Sx,0)=0, \qquad L(Sx,Sy)=L(x,y).} \tag{Compare}$$

For the first equation, induction on the literal equation $R(y,1;H)=2$ uses $H(2)=2$, and then $Z(N(2))=1$. For the second, (Seeds) makes the initial state $F_2(SE(x))$, whose N value is one. For the third, (Seed-rotation), (Iterator-factor), and $N(F_3(u))=N(u)$ give the result directly.

We will also need persistence:

$$L(x,y)=1 \;\longrightarrow\; L(x,Sy)=1. \tag{Persist}$$

Let $r=R(y,E(x);H)$. The Boolean range of $N(r)$ and $0\neq 1$ show from $Z(N(r))=1$ that $N(r)=0$. Thus $H(r)=r$ by (Factor-cases), so the next recursor state is unchanged. No induction on an implication occurs here. Two ordinary equation inductions applied to (Compare) also give $L(n,n)=1$ and $L(Sn,n)=0$.

Every construction above is a fixed finite term in $0,R$. The intended standard meaning $L(x,y)=1$ iff $x\leq y$ explains the design, but neither that order interpretation nor evaluation only at standard numerals is used in the proof. This completes the comparator bootstrap before predecessor, an unrestricted selector, or successor injectivity.

4 Removing the independent successor constructor with one update slot

The successor-seeded one-state proposal in the shared-continuation draft left a precise problem unresolved: construct a term P and prove $P(Sa)=a$ without assuming the ordinary initialized iterator or the unshifted selector that P is intended to recover. The following construction resolves that problem for the ordinary first-order theory. It uses a finite-cycle update with an explicitly accessible transient seed, rather than assuming that an arbitrary initializer can be decoded.

4.1 Exact source theory and the comparator prerequisite

The numerical terms are

$$t ::= x \mid 0 \mid R(n,a;u.h), \qquad Sa := R(0,a;u.0).$$

The update binds exactly one numerical variable u; all other variables are fixed passive parameters. The only computation schemas are

$$R(0,a;u.h)=Sa, \qquad R(Sn,a;u.h)=h[R(n,a;u.h)/u]. \tag{SR}$$

Use ordinary intuitionistic first-order equality logic with genuine falsity, the two boundary sentences

$$\forall x\,(x=0\vee\exists y\,x=Sy), \qquad 0=S0\longrightarrow\bot,$$

and contextual induction on a literal equation:

$$\frac{\Gamma\vdash E(0) \qquad \Gamma,E(z)\vdash E(Sz)}{\Gamma\vdash E(t)}. \tag{EI}$$

The eigenvariable z is absent from the retained context, the conclusion term, and all passive parameters. All descriptions are fixed finite syntax; an equivalent staged first-order signature gives one symbol for

each earlier update description. There is no uniform total evaluator. Denote this ordinary theory by T_{SR}.

Write

$$T(n,a,b) := R(n,a;u.b), \qquad Z(n) := R(n,0;u.0).$$

Their equations are

$$T(0,a,b) = Sa, \quad T(Sn,a,b) = b, \qquad Z(0) = 1, \quad Z(Sn) = 0.$$

Here $1 = S0$. Coverage proves $Z(t) = 0 \vee Z(t) = 1$. Applying Z to $Sx = 0$ gives $0 = 1$, so successor separation is available without successor injectivity.

The preceding comparator construction supplies a finite SR term L with whole-domain proofs of

$$\begin{gathered} L(x,y) = 0 \vee L(x,y) = 1, \qquad L(0,y) = 1, \qquad L(Sx,0) = 0, \\ L(Sx,Sy) = L(x,y), \qquad L(x,y) = 1 \;\longrightarrow\; L(x,Sy) = 1. \end{gathered} \tag{L}$$

Its proof uses only SR, numerical cases, equality, and literal-equation induction. In particular it assumes no predecessor, unshifted selector, pairing, successor injectivity, or induction on compound formulas. This dependency is essential: the familiar initialized-scalar predecessor proof cannot simply be imported before its initializer has been recovered.

4.2 Shifted addition and elementary comparison facts

Define the shifted sum

$$A(a,b) := R(b,a;u.Su).$$

Its computation equations and elementary literal-equation inductions give

$$\begin{gathered} A(a,0) = Sa, \qquad A(a,Sb) = SA(a,b), \\ A(0,b) = Sb, \qquad A(Sa,b) = SA(a,b), \qquad A(a,b) = A(b,a). \end{gathered} \tag{A}$$

For the third equation induct on b; for the fourth do the same with a fixed. Commutativity follows by induction on b, using those two equations for its base and step. These are equations in the source theory, not identities inferred from the intended value $a + b + 1$. An immediate consequence is

$$A(0,1) = 2, \qquad A(Sa,S^2a) = S^2A(a,Sa). \tag{AD}$$

Literal-equation induction on n in the comparator recurrences yields

$$L(n,n) = 1, \qquad L(Sn,n) = 0. \tag{Diagonal}$$

Monotonicity then gives $L(a,Sa) = 1$. Also $L(S^2m,m) = 0$: its Boolean alternative 1 would imply $L(S^2m,Sm) = 1$ by monotonicity, whereas the recurrence and the strict diagonal make this $L(Sm,m) = 0$. Separation rules out $0 = 1$. Finally, monotonicity and the recurrence prove

$$L(St,m) = 1 \;\longrightarrow\; L(t,m) = 1, \tag{Down}$$

because $L(St,Sm) = L(t,m)$. None of these arguments uses successor cancellation or reflection of the comparator into actual equality.

4.3 A reset cycle with an accessible transient seed

For each fixed passive parameter m, define

$$F_m(u) := T\big(Z(L(Su, m)), u, 0\big). \tag{F}$$

This is a finite SR term. Its two Boolean cases are

$$F_m(u) = Su \quad \text{if } L(Su, m) = 1, \qquad F_m(u) = 0 \quad \text{if } L(Su, m) = 0.$$

Thus it increments below the chosen boundary and resets to zero at or beyond that boundary. In particular the preceding comparison facts give

$$F_m(m) = 0, \qquad F_m(Sm) = 0. \tag{Reset}$$

The second equality makes the source's available initializer Sm useful: one explicit application of F_m turns it into zero.

Define two derived trajectories

$$\begin{aligned} J_m(t) &:= F_m\big(R(t, m; u.F_m(u))\big), \\ K_m(t) &:= F_m\big(R(t, m; u.F_m(F_m(u)))\big). \end{aligned} \tag{JK}$$

Their recurrences follow from SR and Reset by direct equational reasoning:

$$\begin{aligned} J_m(0) &= 0, \qquad J_m(St) = F_m(J_m(t)), \\ K_m(0) &= 0, \qquad K_m(St) = F_m(F_m(K_m(t))). \end{aligned} \tag{JR,KR}$$

For K, the outer F_m commutes with the update $F_m \circ F_m$ simply by associativity of substitution. No general iterator-rotation theorem or arbitrary initializer has been assumed. Every induction below holds m fixed until the completed universal identity is subsequently specialized.

4.4 A conditional equation induction, with its literal invariant

The crucial bounded-trajectory statement is

$$L(t, m) = 1 \;\longrightarrow\; J_m(t) = t. \tag{Run}$$

It is proved by induction on the following *single literal equation*:

$$T\big(L(t, m), 0, J_m(t)\big) = T\big(L(t, m), 0, t\big). \tag{Gate}$$

When $L(t, m) = 0$, both sides are 1. When $L(t, m) = 1$, the equation is exactly $J_m(t) = t$. This use of the shifted selector is valid even though the ordinary unshifted selector has not yet been constructed.

At $t = 0$, the comparator value is 1 and $J_m(0) = 0$, so Gate holds. For the successor step retain Gate as the literal induction hypothesis and split the proved Boolean value $L(St, m)$. If it is 0, the new gate equation has both sides equal to 1. If it is 1, Down gives $L(t, m) = 1$; the retained gate equation therefore yields $J_m(t) = t$. The definition of F_m now gives $F_m(t) = St$, and JR gives $J_m(St) = St$. This establishes the new gate equation. EI proves Gate for every t, and its positive Boolean case proves Run.

The retained induction context contains only the displayed gate equation and fixed passive data. Induction on the implication Run, induction on a disjunction, and changing a parameter inside an active induction hypothesis are not used.

At the diagonal, Run and $L(m, m) = 1$ give $J_m(m) = m$. One more computation and Reset consequently give

$$J_m(Sm) = 0. \tag{Period base}$$

4.5 Two ordinary equation inductions give the predecessor

First, with m fixed, induct on the literal equation

$$J_m(A(m,t)) = J_m(t). \tag{Period}$$

At zero, $A(m,0) = Sm$ and both trajectories are zero by Period base. At a successor, $A(m,St) = SA(m,t)$ and JR applies F_m to both sides of the induction equation. This proves Period on the entire source domain; it is not a calculation only at standard counters.

Second, with the same fixed parameter m, induct on

$$K_m(Sa) = J_m(A(a,Sa)). \tag{Double}$$

At zero, both sides are $F_m(F_m(0))$, because $A(0,1) = 2$. At a successor, KR applies F_m twice to the left side, and JR plus AD applies F_m twice to the right side. The induction equation therefore supplies the step.

Now define the single finite SR term

$$P(n) := K_n(n). \tag{P}$$

Its zero equation is $P(0) = K_0(0) = 0$. For arbitrary a, specialize the already proved identities at $m = Sa$ to obtain

$$\begin{aligned} P(Sa) &= K_{Sa}(Sa) \\ &= J_{Sa}(A(a,Sa)) && \text{by Double} \\ &= J_{Sa}(A(Sa,a)) && \text{by commutativity of } A \\ &= J_{Sa}(a) && \text{by Period} \\ &= a && \text{by Run and } L(a,Sa) = 1. \end{aligned}$$

Consequently

$$\boxed{P(0) = 0, \qquad P(Sa) = a.} \tag{Predecessor}$$

The proof has not cancelled a successor or assumed an actual-equality characteristic. It works at arbitrary source elements, including nonstandard elements of any model of the stated ordinary theory. Successor injectivity can now be derived by applying P to an equation $Sa = Sb$.

4.6 Ordinary initialization and both interpretation composites

For any fixed finite update $h(u,\vec{p})$, put

$$\widehat{I}(n,a;u.h) := P\big(R(n,a;v.S(h[Pv/u]))\big), \tag{I}$$

with v fresh. Predecessor and SR give, by direct computation,

$$\widehat{I}(0,a;u.h) = a, \qquad \widehat{I}(Sn,a;u.h) = h[\widehat{I}(n,a;u.h)/u].$$

In particular the ordinary selector is now available as $C(n,a,b) = \widehat{I}(n,a;u.b)$.

The established initialized-scalar reconstruction then applies with all its hypotheses satisfied: ordinary intuitionistic equality logic, coverage and separation, literal-equation induction, successor, and scalar iteration with arbitrary initial values. It proves actual equality reflection, compiles quantifier-free formulas into Boolean terms, recovers contextual quantifier-free induction, and supplies pairing and the compiler for every fixed primitive-recursive description. Its written internal reconstruction, included elsewhere in the report, is required for this conclusion; standard-model term completeness alone would

not suffice. The predecessor step of that older reconstruction is unnecessary here because Predecessor has already been proved.

Conversely, interpret each fixed SR description in ordinary iPRA by primitive recursion with initial value Sa and update h. Then SR, coverage, separation, and the translated literal-equation induction instances are all iPRA theorems. The language uses no function variables or quantification over codes.

To check the stronger return condition, first anchor successor: the zero instance $Sa = R(0, a; u.0)$ translates to ordinary Sa, and its opposite translation is the source Sa. This uses the zero computation, not uniqueness for an as-yet-unidentified recursor. Then compare one fixed description with its double translation after earlier descriptions have been identified. For an SR description both sides have initial value Sa and the same update h; EI on the literal equation comparing their values proves their equality. For an initialized scalar description, the same argument uses its common initial value a. The subsequent primitive-recursion compiler has the same uniqueness argument for its fixed descriptions. In each argument the initializer and passive parameters remain independent formal variables during induction; only after the universally parameterized identity is proved are arbitrary terms substituted for them.

Both interpretation domains are the entire numerical domain, equality is actual equality, and the numerical composite isomorphisms have graph $x = y$. The ordinary logical translations are identities, so external induction on formulas gives the full formula return translations. Thus T_{SR} is equiconsistent and bi-interpretable with full first-order iPRA.

What this proves about simplicity. The ordinary numerical grammar has just zero and the successor-seeded scalar recursor. Section 7 additionally translates this arithmetic proof into the native logical rules, before ordinary connectives or an unshifted selector are available. The Boolean successor decoder in that section removes the circularity that a direct appeal to the established logical reconstruction would have introduced. The numerical identity composites proved here therefore apply to the direct native SR presentation as well.

5 Completion to full iPRA without a standard-model shortcut

This section records the remaining internal arithmetic bridge, adapted from the equality-reflection and primitive-recursion appendices in the supplied corpus [4]. Its input is the predecessor just proved and the derived ordinary iterator $I = \widehat{I}$. Every operation below is consequently a fixed finite SR term. The order of construction is predecessor, actual equality reflection, quantifier-free induction, pairing, and general primitive recursion. We do not assume a general PR compiler in order to construct its prerequisites.

5.1 A characteristic of actual equality

Define

$$C(n, a, b) = I(n, a; u.b), \qquad x + y = I(y, x; u.Su),$$
$$d(x, y) = I(y, x; u.Pu), \qquad \mathrm{sg}(x) = C(x, 0, 1), \qquad W(x) = C(x, 1, Px).$$

They satisfy their ordinary zero/successor equations. Applying P to $Sx = Sy$ proves injectivity. Applying Z to $Sx = 0$ and using separation proves that a successor is nonzero. Coverage now strengthens to $x = 0 \vee x = S(Px)$. Its two branches prove

$$\mathrm{sg}(x) \in \{0, 1\}, \qquad \mathrm{sg}(x) = 0 \iff x = 0,$$
$$W(x) = 0 \iff x = 1, \qquad C(x, a, a) = a.$$

The displayed finite-range notation abbreviates an ordinary disjunction; it is a theorem obtained by cases, not an extra induction schema.

Four literal-equation inductions prove

$$0 + y = y, \qquad (Sx) + y = S(x + y), \qquad d(Sx, Sy) = d(x, y), \qquad d(Sa, a) = 1.$$

The first two induct on y using addition. For the third the base is $P(Sx) = x$, and its step applies P to the current induction equation. For the fourth, induction on a uses the third identity at its step.

With x fixed, one further equation induction on y proves

$$C\big(\mathrm{sg}(d(x, y)), x, y + d(x, y)\big) = x. \tag{Residual}$$

At zero this is $C(\mathrm{sg}(x), x, x) = x$. At a successor put $r = d(x, y)$ and $r' = Pr = d(x, Sy)$. If $r' = 0$, the new selector returns x. Otherwise coverage of r and $P0 = 0$ force $r = Sr'$. The induction equation gives $y + r = x$, while the addition equations give $(Sy) + r' = y + Sr' = y + r = x$. This proves the new selected branch. The invariant being inducted on is exactly the displayed single equation.

It follows that

$$d(Sa, b) = 1 \quad \longleftrightarrow \quad a = b.$$

For the forward direction, Residual gives $b + 1 = Sa$, hence $Sb = Sa$; apply P. For the reverse direction substitute $b = a$ into $d(Sa, a) = 1$. Therefore

$$e(a, b) = \mathrm{sg}(W(d(Sa, b)))$$

has values $0, 1$ and satisfies $e(a, b) = 0 \leftrightarrow a = b$ for the logic's actual equality throughout the source domain. Equality is now decidable: the zero characteristic gives equality, and characteristic one contradicts any alleged equality by separation.

5.2 Every quantifier-free induction instance

For each fixed quantifier-free formula define, by external recursion on its finite syntax,

$$\begin{aligned} b_{s=t} &= e(s, t), & b_{\bot} &= 1, \\ b_{A \wedge B} &= C(b_A, b_B, 1), & b_{A \vee B} &= C(b_A, 0, b_B), \quad b_{A \to B} = C(b_A, b_B, 0). \end{aligned}$$

Induction on formula syntax proves within the theory that b_A has values $0, 1$ and $b_A = 0 \leftrightarrow A$. The conjunction and disjunction cases are the two Boolean alternatives for b_A. For implication, characteristic zero of A supplies A, so the implication has the characteristic of B. Characteristic one of A proves $\neg A$, so the implication is true by intuitionistic explosion and its characteristic is zero. No excluded middle for quantified formulas is used.

Given a contextual induction base and step for $A(n)$, convert them to the base and step for the literal equation $b_A(n) = 0$, use EI, and convert back. Arbitrary retained quantified assumptions are permitted. This obtains all quantifier-free induction in the scalar language.

5.3 Pairing and the general recursion compiler

The familiar semiring and linear-order identities now follow by quantifier-free inductions from addition, predecessor, and monus. In particular define $a \leq b$ by $d(a, b) = 0$ and $a \cdot b = I(b, 0; u.u + a)$. All these are scalar terms.

For fixed n set

$$U_n(v) = C(d((Sv)^2, n), Sv, v), \qquad B_n(i) = I(i, 0; v.U_n(v)), \qquad q(n) = B_n(n).$$

Quantifier-free induction on i, keeping n fixed, gives the simultaneous invariant

$$B_n(i) \le i, \quad B_n(i)^2 \le n, \quad B_n(i) = i \ \vee \ n < (SB_n(i))^2.$$

Its base is immediate. If the update increments, its test rules out the strict-upper-bound alternative, so the old state is i and the new state is Si; the square bound is the test. If it does not increment, the strict upper square bound holds and the old state is retained. At $i = n$, the alternative $B_n(n) = n$ also implies the strict upper square bound, since $n < (n+1)^2$. Consequently

$$q(n)^2 \le n < (q(n)+1)^2.$$

Monotonicity of squares proves uniqueness of this root interval.

Define

$$\pi(a, b) = (a + b)^2 + a, \quad \ell(n) = d(n, q(n)^2), \quad r(n) = d(q(n), \ell(n)).$$

For $k = a + b$, $k^2 \le k^2 + a < (k+1)^2$ because $a \le k$. Root uniqueness gives $q(\pi(a, b)) = a + b$, and subtraction gives

$$\ell(\pi(a, b)) = a, \qquad r(\pi(a, b)) = b.$$

Surjectivity of π is unnecessary; the domain is never restricted to its range.

Suppose the earlier descriptions $g(\vec{p})$ and $h(n, z, \vec{p})$ have already been compiled. Put

$$V(n, \vec{p}) = I\big(n, \pi(0, g(\vec{p})); u.\pi(S\ell(u), h(\ell(u), r(u), \vec{p}))\big), \qquad f(n, \vec{p}) = r(V(n, \vec{p})).$$

Equation induction proves $\ell(V(n, \vec{p})) = n$. The two projection identities then give

$$f(0, \vec{p}) = g(\vec{p}), \qquad f(Sn, \vec{p}) = h(n, f(n, \vec{p}), \vec{p}).$$

Initial functions and composition are immediate. External induction on each fixed finite PR description therefore supplies its SR term and internal defining-equation proofs. Every translated quantifier-free induction instance is available by the preceding reflection argument.

5.4 Both return interpretations

Interpret SR in iPRA by primitive recursion with seed Sa, and interpret each PR description in SR by the displayed compiler. In either composite, first identify successor: $Sa = R(0, a; u.0)$ maps to ordinary Sa by the zero clause, and compiling that successor back gives the source Sa. This anchors the successor operation before uniqueness is invoked. Then, compare a fixed operation with its double translation after its earlier subdescriptions have been identified. Their bases and updates agree; equation induction on the equality of their values proves their agreement at every argument. Keep initializer and passive parameters independent throughout that induction, and substitute dependent terms only afterward. This applies equally to SR and ordinary PR recursion.

Both domains are the entire original numerical domain, defined by $x = x$; both equality relations are actual equality; and each definable numerical isomorphism has graph $x = y$. The ordinary connectives, falsity, and quantifiers are preserved, so formula induction gives the full composites. This establishes bi-interpretability, not merely mutual interpretation or standard-model agreement. Effective finite proof translations preserve contradiction in both directions and therefore establish equiconsistency.

6 The native checker comparison, with its exact boundary

The principal program in Appendix A directly checks native successor-seeded scalar recursion, SR0/SRS. Its numerical grammar is $x, 0, R(n, a; u.h)$, with a serialized unary node for the fixed abbreviation $Sa = R(0, a; u.0)$. The auxiliary I profiles instead supply initialized iteration, and their independent source changes are described below. Its formulas are equations, $Q_y B$, and $D_{x,y} B$, with ordinary meanings

$$Q_y B = B(0) \to \forall y B(y), \qquad D_{x,y} B = \exists x (B(x, 0) \to \forall y B(x, y)).$$

The designated native absurdity is $0 = S0$.

6.1 Its entire primitive inventory

Hypothesis use and transport are

$$\frac{}{\Gamma \vdash A}\ (A \in \Gamma), \qquad \frac{\Gamma \vdash s = t \quad \Gamma \vdash F(s)}{\Gamma \vdash F(t)}.$$

Here F may be any native formula. The closure and paired eliminations are

$$\frac{\Gamma \vdash Q_y B \quad \Gamma \vdash B(0)}{\Gamma \vdash B(t)}, \qquad \frac{\Gamma \vdash D_{x,y} B \quad \Gamma, Q_y B(a, y) \vdash A}{\Gamma \vdash A}.$$

The eigenparameter a is fresh for the conclusion, context, and passive matrix parameters. The two permitted paired introductions are

$$\frac{\Gamma, B(t, 0) \vdash B(t, Sz)}{\Gamma \vdash D_{x,y} B}, \qquad \frac{\Gamma, E(t, z) \vdash E(t, Sz)}{\Gamma \vdash D_{x,y} E}.$$

The second is allowed only for a literal equation E; z is fresh for all retained data. The local hypotheses are discharged. Compound matrices have only the first, case-style introduction.

Finally the computation-supply rule is

$$\frac{\Gamma, H(w) \vdash A}{\Gamma \vdash A}, \qquad w \notin \mathrm{FV}(\Gamma, A), \qquad H(0) \text{ is a computation theorem.}$$

The implementation generates only the specified SR0/SRS instances (I0/IS in the auxiliary profiles) and their zero-extension aliases; it has no general theorem recognizer. This rule is justified by substituting $w := 0$ into the premise and cutting its known instance. Conversely an ordinary computation leaf is supplied as a constant-in-w hypothesis and closed by Hyp. Neutral unmatched Eq/Q requests add only the theorem $0 = 0$ after their major has been proved. There is no additional logical axiom or unrestricted induction rule.

6.2 Why this is a full intuitionistic presentation

Section 7 derives predecessor in native SR before using the present subsection. Its initialized-iterator macro then gives the unshifted selector $C(n, a, b) = I(n, a; u.b)$ by computation. On equations define the gate and selector

$$G_v(s = t) :\ C(v, 0, s) = C(v, 0, t), \qquad M_v(s = t, r = q) :\ C(v, s, r) = C(v, t, q).$$

Extend gates through D binders. A Q formula is interderivable with a D formula with unused existential variable, by case introduction, paired elimination, and closure specialization. The padding equivalence

$A \dashv\vdash D_{u,y}G_yA$ follows from the true zero gate and its successor instance. Pad the shallower operand of a selector until its binders align, then extend the selector through the aligned binders. Induction on this finite syntax gives

$$\vdash G_0A, \quad G_{Sv}A \dashv\vdash A, \qquad M_0(A,B) \dashv\vdash A, \quad M_{Sv}(A,B) \dashv\vdash B.$$

For $p(z) = I(z, 0; u.C(u, 1, 0))$, finite computations give $p(1) = 1, p(2) = 0$. The ordinary connectives and quantifiers have the macros

$$\begin{aligned} A \Rightarrow_* B &= D_{u,z}M_z(A,B), \\ A \wedge_* B &= D_{u,z}G_zM_{p(z)}(A,B), \\ \exists_* xA(x) &= D_{x,y}G_yA(x), \\ A \vee_* B &= \exists_* xM_x(A,B), \\ \forall_* xA(x) &= A(0) \wedge_* D_{u,z}G_zA(z), \\ \bot_* &= (0 = 1). \end{aligned}$$

All unused binders and bound-name changes are fresh. The selector endpoints prove the implication rules; the gates prove the existential rules; numerical cases prove disjunction elimination. The instances 1 and 2 give the conjunction projections. Numerical coverage, obtained by cases, gives universal elimination; the zero instance and successor cases give universal introduction. Under $0 = 1$, transport through C proves any equation, and case introduction by formula induction proves explosion. Progress introduction gives contextual equation induction. No induction on a quantified formula is introduced.

The nontrivial native return formulas are

$$Q_yB \dashv\vdash B(0) \Rightarrow_* \forall_* yB(y), \qquad D_{x,y}B \dashv\vdash \exists_* x\big(B(x,0) \Rightarrow_* \forall_* yB(x,y)\big).$$

Forward, open the paired formula and use its closure under an assumed base. Backward, eliminate the displayed existential and use case introduction, with implication and universal elimination providing its successor premise. These arguments cover arbitrary raw native matrices.

For arithmetic, interpret SR in ordinary iPRA by $R(n, a; h) = I(n, Sa; h)$. Conversely the native bootstrap below derives the predecessor and initialized iterator; the present logical reconstruction then applies, followed by Section 5. Recursion uniqueness supplies the numerical identity composites, with successor anchored by SR0 first. The displayed Q/D return equivalences supply the native-formula composites. Thus both measured native presentations have full equivalence; the direct SR checker needs no uncharged translation to an I checker.

6.3 Codec and scope

Input is one JSON array. Its last entry is the requested closed native formula; earlier four-field commands $[m, s, u, t]$ are popped backward. An unused prefix is inert. Success prints ACCEPT and exits zero; failure prints REJECT and exits nonzero, possibly with a traceback.

The sorted tree decoder uses arity 0 for numerical zero/formula truth, arity 1 for successor/Q, arity 2 for annotated numerical zero/equality, and arity 3 for the scalar recursor/D. Its depth increments are respectively immaterial, 2, 1, and 4. At numerical depth $2q+1$ there are q protected numerical slots; the positive coordinates $2j-1, 2j$ alias the same slot. Successor adds a dummy zero; a recursor's counter and seed add two dummy zeros, whereas its update gets two aliases of one state. The two unused fields of a D node are checked annotations. The global negative floor decreases at each continuation, introducing a rigid fresh parameter. The opener checks images at numerical root; its structural induction gives capture-free substitution and scope safety. Arrays of arity greater than three reject.

The old goal is checked before fresh allocation. A matched transport or closure minor is likewise checked before allocation. Arithmetic eagerly checks the recursive predecessor term, even if its update ignores the state. The generated equation's zero instance is an SR0 or SRS equation (I0 or IS for the auxiliary profiles) after dummy zero slots are identified. Freshness plus known-instance cut therefore discharges its continuation. These are mathematical guards, not optional input hygiene. The supplied mutation controls demonstrate false acceptance if any of major checking, predecessor validation, or the equation-only induction restriction is removed.

6.4 What changed in the measured programs

The direct SR byte profile additionally replaces the zero-counter result `t` by `[t]`, the serialized successor. This changes the checked arithmetic theory exactly from initialized I to SR; the nonzero computation, validation, freshness, and logical dispatch are unchanged. A unary successor remains a fixed syntactic abbreviation and is fully charged as syntax in the implementation.

The auxiliary I byte profile replaces one conditional assignment statement by its equivalent conditional expression. Evaluation order, lazy selection, and all accepted inputs remain exactly the same. This saves six normalized bytes and increases the other two measures. It is a small source refactoring.

The structural profile removes the redundant list concatenation `a + []` from the existing exception-based traversal. Canonical array/integer certificates are unchanged. The additional successful traversed JSON aliases have the following explicit erasure:

$$\texttt{""} \mapsto [], \qquad \texttt{\{\}} \mapsto [], \qquad \texttt{\{"": value\}} \mapsto [[]].$$

The ignored object value has no numerical or logical meaning. Nonempty strings recurse into nonempty characters and fail; objects with any nonempty key fail for the same reason. A dictionary cannot survive the major-premise slice or the DI goal's integer lookup. Empty aliases are zero counters; the one-key alias is successor zero, whose starred predecessor is the empty-string zero and is eagerly normalized. Every successfully appended context entry remains a list.

Soundness is proved by decoding each successful inference step with these aliases. Do not erase an entire certificate and claim operational equivalence: erasure can change a neutral mismatch into a matched elimination. The direct decoded proof still uses exactly the sound logical and arithmetic rules above. Canonical completeness is unchanged because canonical certificates have the original array/integer form. This is a serialization simplification, not a new logical principle.

6.5 An optional predecessor-specified computation interface

There is also an initialized-iterator profile below all three attained incumbent coordinates. Its complete source measures 280 normalized Python tokens, 314 AST nodes, and 684 normalized bytes, compared with the attained coordinate envelope 281/315/685. This is a narrow implementation improvement, not a new arithmetic basis. It changes the computation packet protocol.

An arithmetic packet supplies the triple $s = [n, a, r]$. If n is false, its generated equation is $I(0, a; r) = a$. If n is true, the checker *constructs* the successor counter and generates

$$I(Sn, a; r) = r[I(n, a; r)/u].$$

Thus there is no submitted alleged successor constructor whose arity must be tested. The relevant source is

```
n, t, u = s
h = [[n and [n], t, u], i(u, s, 1) if n else t]
```

The update's used recursive image is still checked at numerical root. When the update ignores that image, its value is independent of the recursive state and no such check is required. The generated source is checked by the common opening in every successful continuation.

Zero-instance soundness. For false n, every surviving scalar/empty alias denotes zero. The seed in the source and its exposed copy agree after the newly allocated name is replaced by zero, by the same zero-extension induction as before. For true n, the source counter is literally the singleton $[n]$; therefore its numerical interpretation is a successor, regardless of whether n itself is atomic, unary, or an iterator. If the update uses its state, the opener validates the entire recursive source s before inserting it. Extra zero slots under the displayed successor do not change the decoded predecessor. If the state is not used, the right side is the same update independent of that state. The old parameters remain fixed, and every permitted newly exposed name disappears at the zero instance. Consequently the generated assumption $H(w)$ has a proved instance $H(0)$ in both branches. The known-instance cut discharges it. This argument must be read with the explicit sorted codec, not as an assertion that every raw packet is a standard first-order term.

Canonical completeness at predecessor zero. Canonical terms other than literal zero have truthy encodings. To obtain an ordinary successor computation whose predecessor is literally zero, put

$$z = I(0, 0; u.0), \qquad \vdash z = 0.$$

The encoding of z is nonempty. Hence the new packet generates the successor computation at z. Transport that whole equation along $z = 0$ with the motive

$$F(v) \equiv I(Sv, a; u.r) = r[I(v, a; u.r)/u].$$

This gives $F(0)$. The proof of $z = 0$ uses the base packet, so the expansion is finite and noncircular. All other native inference rules are unchanged. Replacing each computation leaf in a native derivation by these finite expansions proves canonical completeness. The new serializer performs this expansion in named syntax and then assigns fresh labels afresh.

Small source conventions. The printable lookup table uses the identities $\mathrm{ASCII}(B) \bmod 8 = 2$, $\mathrm{ASCII}(A) \bmod 8 = 1$, and $\mathrm{ASCII}(D) \bmod 8 = 4$. Constructor arities and their scope increments remain exactly the same. The default image `0j` is an invalid-image marker: like `None`, it cannot pass comparison with an integer, and a used missing image therefore rejects. JSON cannot supply a complex literal. Only the standard `json` module is imported; the wildcard import changes no trusted mathematical dependency.

The Boolean expression replacing conditional transport also preserves successful behavior. Opening a nonempty formula tree preserves its outer arity. Therefore if a successful formula opening $i(t, s)$ is false, both openings $i(t, s)$ and $i(t, u)$ are the same empty tree or scalar truth. In the matched case the latter equals the requested goal a. The fallback to a is thus exactly the same result as the conditional minor, including that exceptional false-value case.

The readable checker uses the direct table $(0, 2, 1, 4)$, the ordinary `None` marker, and an explicit conditional. It implements the same successful semantics and is supplied for review. These source conventions account for the final one-unit margins; they should not be presented as mathematical breakthroughs.

6.6 Evidence and its limits

All 248 inherited-certificate and malformed-input CLI runs passed, as did 24 alias-specific runs and three guard-removal mutation controls. The direct SR checker passed 52 normal/optimized runs on 26 certificates, including actual native Boolean-decoder and restricted-implication proofs. The jointly smaller I profile and its readable version passed 136 runs on 34 new-protocol cases. The SR source-macro evaluators passed 1279 comparator checks and 926 predecessor/initializer checks, for 2,205 arithmetic transcription checks. The independent bounded-coordinate compiler passed its 294 finite checks. The complete-source measurements reproduced the table in Section 1. The executable examples cover actual use of induction hypotheses, nonvacuous witnesses, transport below a binder, generated zero witnesses, scope failures, malformed counters, failed majors, and forbidden compound induction. Finite evaluations of the SR macros check the numerical transcription, while the arbitrary-element theorems rest on the written induction proofs. The entire metatheory has not been formalized in Lean, Rocq, or another proof assistant.

7 The native SR bootstrap without an unshifted selector

The ordinary SR proof can be carried out in the native Choice-DI calculus before ordinary logical connectives have been reconstructed. The missing ingredient is a restricted implication whose antecedent is an equation between Boolean terms. Such an antecedent permits successor cancellation by a finite decoder; general successor cancellation is not assumed. This supplies precisely the contextual finite case analyses used in the arithmetic proof.

7.1 Primitive setting and the two immediate derived rules

Use the stated native calculus with SR0/SRS, retaining its Eq/Q/D formulas, hypotheses, equality transport, paired and closure elimination, and both case and literal-equation progress introductions. Successor is the numerical abbreviation $Sa = R(0, a; u.0)$. All rules in this section are syntactic derivability statements in that native calculus; no ordinary disjunction elimination is presumed.

For any native formula $B(y)$ the native rules derive unconditional numerical cases

$$\frac{\Gamma \vdash B(0) \qquad \Gamma \vdash B(Sz)}{\Gamma \vdash B(t)}, \qquad z \text{ fresh for retained data.} \tag{NC}$$

Indeed case introduction with an unused witness coordinate constructs $D_{x,y}B(y)$ using the successor proof and ignoring its local base assumption. Paired elimination supplies $Q_yB(y)$, and closure specialization with the given base proof supplies $B(t)$. The analogous use of progress introduction gives contextual induction on a literal equation, with the same retained-parameter restrictions.

Reflexivity requires no injectivity: the fixed computation $R(1, 0; u.t) = t$ can be used both as the major and the minor of transport with motive $v = t$, yielding $t = t$. Symmetry, transitivity, and numerical congruence follow by the usual explicit equality motives.

Let $T(n, a, b) = R(n, a; u.b)$ and $Z(n) = T(n, 0, 0)$, as in the arithmetic construction. Under $0 = 1$, transport in the control coordinate gives $T(0, 0, s) = T(1, 0, s)$, hence $1 = s$, for every s. Thus every equation follows from the designated absurdity. This atomic explosion is enough for all arithmetic arguments below. It does not use a selector with unshifted endpoints.

7.2 A finite successor decoder, before comparison

The equations for Z immediately give, by NC on r,

$$Z(Z(Zr)) = Zr. \tag{Z3}$$

Define $N(n) = Z(R(n, 0; Z))$. Direct computation and Z3 give

$$N(0) = 0, \qquad N(Sn) = Z(N(n)), \qquad N(S^2 n) = N(n).$$

NC on r, using $N(0) = 0$ and $N(1) = 1$, also proves

$$N(Zr) = Zr.$$

Consequently the finite decoder

$$p_2(w) := N(Sw)$$

satisfies the unconditional identity

$$p_2(S(Zr)) = Zr. \tag{BD}$$

All these proofs are equation computations and unconditional NC. Neither a Boolean-range disjunction nor contextual Boolean case elimination was used to obtain BD.

In what follows a *Boolean term* means syntactically a term Zr, or a fixed numeral $0, 1$ identified with $Z1, Z0$. Both $N(n)$ and the eventual comparator $L(x, y)$ are Boolean terms in this strict sense. For Boolean b, c, applying p_2 to $Sb = Sc$ yields $b = c$ by BD. Conversely congruence gives $Sb = Sc$ from $b = c$. This is a local finite decoder, not general successor injectivity.

7.3 Restricted implication and genuine contextual Boolean cases

For Boolean b, c and an arbitrary equation E written $s = t$, define the following native formula, with fresh binders and unused x:

$$\mathcal{I}(b = c, E) := D_{x,y}\big(T(y, b, s) = T(y, c, t)\big). \tag{BI}$$

Its zero matrix is $Sb = Sc$, equivalent to $b = c$ by BD; its successor matrix is exactly E by computation. The native rules therefore derive the following two rules:

$$\frac{\Gamma, b = c \vdash E}{\Gamma \vdash \mathcal{I}(b = c, E)}, \qquad \frac{\Gamma \vdash \mathcal{I}(b = c, E) \qquad \Gamma \vdash b = c}{\Gamma \vdash E}. \tag{BII,BIE}$$

For introduction, choose an arbitrary unused witness, for instance zero. The retained zero matrix $Sb = Sc$ supplies $b = c$ after applying p_2. Insert the given premise derivation and rewrite its conclusion into the successor matrix before closing case introduction. For elimination, paired elimination supplies the closure of that matrix. Congruence turns the given $b = c$ into its zero instance; specialization at 1 then gives E. The opened witness is unused and fresh. Ordinary assumption cut here means finite substitution of derivations, admissible by induction on native proof trees.

Now let $b = Zr$ be a fixed Boolean term. For an arbitrary retained native context Γ and an equation E, the following contextual case rule is derivable:

$$\frac{\Gamma, b = 0 \vdash E \qquad \Gamma, b = 1 \vdash E}{\Gamma \vdash E}. \tag{BC}$$

Choose a new numerical variable w absent from Γ, b, E, r, and apply unconditional NC to the native formula

$$B(w) := \mathcal{I}(b = Zw, E).$$

Its zero proof is BII applied to the supplied $b = 1$ branch, since $Z0 = 1$. Its successor proof is BII applied to the supplied $b = 0$ branch, since $Z(Sz) = 0$. NC gives $B(r)$. Its antecedent $b = Zr$ is reflexive, so BIE gives E.

The retained context does not change during this argument. In particular it may contain a literal induction hypothesis mentioning the original counter and the Boolean term b. The varying variable w is fresh, distinct from that original counter. This is the precise reason BC can replace the ordinary proof's Boolean case analyses; simply applying NC to a context that depends on its case variable would have been invalid.

The consequent of $\mathcal{I}$ need only be an equation. Although $B(w)$ is compound, unconditional NC is available for every native formula. No selector for arbitrary formulas, unrestricted implication, or trit case-elimination rule is required.

7.4 The modulo-three cycle without a range disjunction

Use the same numerical definitions

$$j(u) = T\big(u, 0, T(Z(N(u)), 1, 0)\big), \qquad M(n) = j(j(R(n, 0; j))).$$

The fixed computations $j(0) = 1$, $j(1) = 2$, $j(2) = 0$ use only the already established initial values of N. Instead of importing a logical statement that the range of j is three-valued, prove the single equation

$$j^3(j(u)) = j(u) \tag{J3}$$

by unconditional NC on u. At zero it is a fixed cycle computation. At $u = Sz$, invoke BC on the Boolean term $N(Sz)$. When its value is zero, the inner selector gives $j(Sz) = 0$; when its value is one, it gives $j(Sz) = 2$. In both cases J3 is a fixed cycle computation. The goals of this Boolean case use are equations.

Direct unfolding proves $M(0) = 0$ and $M(Sn) = j(M(n))$. Since $M(n)$ is itself of the form $j(v)$, J3 gives

$$M(S^3 n) = M(n).$$

This obtains the exact modulo-three equation used by the remaining construction without assuming a native three-way disjunction or a successor decoder on arbitrary numbers.

7.5 Transferring the arithmetic proof, step by step

Every subsequent step of the comparator and predecessor proofs now has a native derivation with the same numerical terms. The dependencies are as follows.

1. The doubling and tripling residue invariants are literal equations. Their successor steps rewrite the update directly with their respective induction equations $N(d_2(n)) = 0$ and $M(d_3(n)) = 0$. They use the two- and three-period equations just proved, and need no range cases.

2. The ramp invariant $M(G(n)) = N(n)$ and the two scaling identities use Boolean case splits only on $N(n)$. Replace each of those ordinary splits by BC; retain the original literal induction equation in Γ. The finite update values at residues zero and one are direct computations. The unused residue-two reset is also a direct fixed computation. No trit elimination rule is needed.

3. The factor-map identities split only $N(u)$. Seed rotation, iterator factorization, and the zero row of comparison are equation inductions with fixed passive parameters. The comparator recurrences are therefore native equation theorems.

4. Comparison persistence is a contextual equational derivation from the premise $L(x, y) = 1$. Split the Boolean $N(R(y, E(x); H))$ by BC. Its value one contradicts that retained premise by fixed computation; atomic explosion handles this branch. Its value zero makes H fix the state and proves the next comparison equation. Other conditional comparison facts use the same pattern, with Boolean L terms.

5. The bounded-cycle run is obtained by induction on the same literal gate equation

$$T(L(t, m), 0, J_m(t)) = T(L(t, m), 0, t).$$

   Its successor proof uses BC on $L(St, m)$ while retaining that literal induction equation. The downward comparison fact is used as its already established contextual equation derivation. The period and doubled-trajectory arguments are then pure equation inductions. Consequently the native calculus proves $P(0) = 0$ and $P(Sa) = a$.

This is a finite proof transformation: replace each recorded Boolean case with the displayed BC derivation, replace any impossible Boolean branch with native atomic explosion, and use the native derived EI for each displayed literal induction. All remaining steps are computation, equality transport, substitution, or unconditional NC. No part of the proof asks for induction on a native compound formula.

7.6 Closing the full logical and arithmetic equivalence

With the native predecessor theorem available, the finite macro

$$I(n, a; u.h) := P\big(R(n, a; v.S(h[Pv/u]))\big)$$

has the ordinary initialized-iterator equations by native computation and transport. In particular $C(n, a, b) = I(n, a; u.b)$ is now an unshifted selector. At this point the established Choice-DI gate, padding, formula-selector, and logical-macro reconstruction applies without circularity. It gives the complete intuitionistic first-order logical interface, coverage and separation, and the native formula return translations. The ordinary SR arithmetic and composite-uniqueness proofs then apply to this reconstructed interface.

Conversely every native SR rule is sound in the ordinary SR theory under the usual readings of Q and D: case introduction uses numerical coverage, and progress introduction uses only literal-equation induction. The domains and numerical equality are unchanged, and the familiar Q/D return equivalences are available after C has been derived. Thus the direct native SR calculus itself is equiconsistent and bi-interpretable with full first-order iPRA. A serialized successor node may be retained as the charged abbreviation $R(0, a; u.0)$; its presence is an implementation convenience, not a new numerical axiom.

8 A bounded-coordinate obstruction for a tied-seed recursor

One tempting way to remove a numerical constructor field is to make the initializer a special evaluation of the update itself. The following obstruction shows that this particular reduction loses an essential term-forming ability. It is an obstruction to a proposed reduction, not an impossibility theorem for every smaller presentation of iPRA.

8.1 The candidate and its standard interpretation

Consider finite numerical terms

$$t ::= x \mid 0 \mid B(n; u, v.h),$$

where only h binds the distinct old-counter variable u and previous-state variable v. Every update is fixed finite earlier syntax; its other variables are passive numerical parameters. Define

$$Sa := B(0; u, v.a)$$

when u, v are fresh for a, and propose the two computation schemas

$$B(0; u, v.h) = S(h[0/u, 0/v]), \qquad B(Sn; u, v.h) = h[n/u, B(n; u, v.h)/v]. \tag{B}$$

In the standard natural-number interpretation these mean, for each fixed previously interpreted function h,

$$B_h(0, \vec{p}) = 1 + h(0, 0, \vec{p}), \qquad B_h(m+1, \vec{p}) = h(m, B_h(m, \vec{p}), \vec{p}). \tag{$B_{\mathbb{N}}$}$$

Thus the displayed abbreviation S has exactly its ordinary successor meaning. Structural recursion on descriptions followed by primitive recursion on m supplies the standard interpretation of every term. There is no object-language quantification over functions.

Several useful operations survive. Put

$$D(n, a) := B(n; u, v.a), \qquad W(n) := B(n; u, v.u), \qquad P(n) := W(D(n, n)).$$

The computation schemas give, without induction,

$$D(0, a) = Sa, \quad D(Sn, a) = a, \qquad W(0) = 1, \quad W(Sn) = n,$$
$$P(0) = 0, \qquad P(Sn) = n.$$

In particular, loss of an immediate predecessor is not the difficulty with this proposal.

8.2 A property of every numerical term

For a function $f : \mathbb{N}^r \to \mathbb{N}$, say that coordinate i is a *bounding coordinate* if there is a nondecreasing function $F : \mathbb{N} \to \mathbb{N}$ such that

$$f(x_1, \ldots, x_r) \leq F(x_i) \quad \text{for every } \vec{x}. \tag{BC}$$

Equivalently, for every fixed bound b, its image on $x_i \leq b$ is bounded independently of all the other coordinates. A globally bounded function is allowed as a separate case, including when $r = 0$. The bound need not be a term of the candidate language. In the proof below it can always be chosen primitive recursive.

Bounded-coordinate theorem. Every term in the $0, B$ language has a standard interpretation which is either globally bounded or has a bounding coordinate among its free numerical variables. A coordinate and a nondecreasing primitive-recursive bound can be obtained compositionally from its finite syntax.

Proof. Use structural induction, treating the bound variables of an update as additional independent input coordinates when processing its body. A variable x_i has bound $F(z) = z$ in coordinate i, and 0 is globally bounded by 0.

For the inductive step write $t = B(n; u, v.h)$ and let H be the bound already obtained for h, whose free variables are among $u, v, \vec{p}$. Bound names are first made fresh for n and the passive parameters. There are four cases.

1. If h is globally bounded by a constant K, its initial value is at most $K + 1$ and each subsequent value is at most K. Consequently $B_h(m, \vec{p}) \le K + 1$ for every $m, \vec{p}$.

2. Suppose a passive parameter p_j is the chosen bounding coordinate of h. Both $h(0, 0, \vec{p})$ and every $h(m, B_h(m, \vec{p}), \vec{p})$ are at most $H(p_j)$, uniformly in the counter and previous state. Therefore

$$B_h(m, \vec{p}) \le H(p_j) + 1.$$

   This remains true when the counter term n also depends on p_j or on any of the other passive parameters: the estimate is independent of its value.

3. Suppose u is the chosen bounding coordinate of h. Then $h(u, v, \vec{p}) \le H(u)$ uniformly in $v, \vec{p}$. Define the nondecreasing primitive-recursive function

$$K(m) := 1 + \max_{0 \le q \le m} H(q).$$

   The base value is at most $H(0) + 1 \le K(0)$; at every positive counter $m + 1$ the value is at most $H(m) \le K(m + 1)$. Hence $B_h(m, \vec{p}) \le K(m)$ for all arguments.

4. Suppose v is the chosen bounding coordinate of h. Then $h(u, v, \vec{p}) \le H(v)$ uniformly in $u, \vec{p}$. Define

$$K(0) := H(0) + 1, \qquad K(m + 1) := \max\{K(m), H(K(m))\}.$$

   The initial value of B_h is at most $H(0) + 1$ because its previous-state argument in $h(0, 0, \vec{p})$ is exactly zero. If $B_h(m, \vec{p}) \le K(m)$, monotonicity of H gives

$$B_h(m + 1, \vec{p}) \le H(B_h(m, \vec{p})) \le H(K(m)) \le K(m + 1).$$

   Ordinary metatheoretic induction on the standard natural number m therefore gives $B_h(m, \vec{p}) \le K(m)$. The sequence K is nondecreasing and primitive recursive.

In the last two cases the term is bounded by $K(n(\vec{p}))$. Apply the structural induction hypothesis to the counter term n. If n is globally bounded by N, then $K(N)$ is a constant bound for the whole term. Otherwise $n(\vec{p}) \le N(p_i)$ for a chosen coordinate p_i and a nondecreasing primitive-recursive function N; then $K(N(p_i))$ is the required bound. This exhausts the possible bounding coordinates of h and completes the proof. □

This induction processes nested occurrences of B and arbitrary repeated passive variables. Capture-free substitution does not escape the result: if an already bounded expression is composed with finitely many terms, apply the bound to its chosen argument term and then use that term's chosen bound. Unused arguments cannot create a second unbounded input. Finite macro expansion, including all occurrences of S, D, W, P, therefore preserves the theorem. The argument is about standard values and does not assume that this metatheoretic bound is itself provable in a weak candidate theory.

8.3 What the obstruction rules out

No ordinary two-branch selector. There is no $0, B$ term with standard values

$$C(0, a, b) = a, \qquad C(n + 1, a, b) = b.$$

Such a function is not globally bounded. The control coordinate n cannot bound it, since $n = 0$ leaves a arbitrary. The coordinate a cannot bound it, since $a = 0, n = 1$ leaves b arbitrary. The coordinate b cannot bound it, since $b = 0, n = 0$ leaves a arbitrary. These are all three input coordinates, contradicting the theorem.

No ordinary addition or a numerically injective pairing term. Addition $a+b$ has no bounding coordinate: fixing either argument at zero leaves the other unbounded. More generally, no binary term can be an injection $\mathbb{N}^2 \to \mathbb{N}$. If one coordinate bounds such a term, fixing it at zero confines infinitely many distinct pairs to a finite set of output values; a globally bounded term fails injectivity for the same reason.

A one-sided selector does not repair the problem. If literal-equation induction is available, the candidate does define the one-sided mask

$$M(n, a) := P\big(P(B(a; u, v.D(n, v)))\big), \qquad M(0, a) = a, \quad M(Sn, a) = 0.$$

For $n = 0$ its update is successor and its initial value is 2; equation induction gives $B(a; u, v.D(0, v)) = S^2a$. For $n = Sx$ its update is the identity and its initial value is 1; equation induction gives $B(a; u, v.D(Sx, v)) = 1$. The two predecessor equations then establish the mask endpoints. This causes no contradiction: a is a bounding coordinate of M. Combining two such masks into an unrestricted selector is precisely an operation that the theorem excludes in this term language.

8.4 Exact logical scope

The obstruction applies to standard-model *term representation*. Adding sound logical rules or sound induction principles cannot make a term compute addition, selection, or an injective numerical pairing in $\mathbb{N}$: the semantic obstruction is independent of such proofs. In particular this proposal cannot justify a direct compiler from the old scalar-iterator language, because that language already has the ordinary selector $I(n, a; u.b)$.

It does not prove that a theory using this numerical syntax cannot be equiconsistent or bi-interpretable with iPRA. The user explicitly permits other domains and languages, and an interpretation may use definable relations, quotients, several coordinates, or functions defined by unique-existence formulas rather than numerical terms. No exclusion of all such interpretations is claimed. Nor does the result address the different successor-seeded recursor with an *independent initializer* $R(n, a; u, v.h)$: that initializer invalidates the bounded-coordinate induction and is exactly the field whose removal was being considered.

The positive methodological conclusion is concrete: eliminating an initializer by setting it to $1 + h(0, 0)$ is not a sound route to a term-equivalent arithmetic basis, even though it retains successor, ordinary predecessor, and a useful one-sided mask.

9 Why deleting atomic case introduction is not an equivalence

Consider the native Eq/D calculus with closure clauses Q. Keep arithmetic computation, equality transport, paired elimination, closure specialization, and identity/availability. Modify paired introduction so that its local assumption is always $E(t,z)$ when the matrix is a literal equation, and always $B(t,0)$ otherwise. This removes the independent case/progress choice for atomic matrices.

There are two sound interpretations on standard natural-number arithmetic. In the first, every D has its usual closure reading

$$D_{x,y}B = \exists x\big(B(x,0) \to \forall y\, B(x,y)\big).$$

In the second, interpret literal equations by actual equality, interpret Q_yB by $B(0) \to \forall y B(y)$, and define D recursively by

$$(D_{x,y}B)^\dagger = \begin{cases} \exists x\, \forall y\big(B(x,y) \to B(x,Sy)\big), & B \text{ a literal equation,} \\ \exists x\big(B^\dagger(x,0) \to \forall y B^\dagger(x,y)\big), & B \text{ compound.} \end{cases}$$

Both interpretations validate every remaining rule. In particular, an atomic progress witness supplies its closure clause by ordinary induction on the standard natural numbers, so paired elimination is sound. Compound case introduction is sound because every positive natural number is a successor. Equality transport preserves the literal/compound classification and respects the displayed interpretations.

Let $E(y)$ be $y = 1$, with an unused existential variable x. In the first interpretation, $D_{x,y}E(y)$ is true because $E(0)$ is false. In the second interpretation it is false, because $E(1)$ is true and $E(2)$ is false. The original calculus proves this formula by atomic case introduction: under its base assumption $0 = 1$, explosion supplies the required successor equation. Hence the restricted calculus is not equivalent to the original native calculus.

Padding an equation into a compound formula may recover an ordinary logical interface, but this does not repair the missing native round trip. If atomic D is decoded as closure, its closure formula cannot be translated back to the native D in the second interpretation. If it is decoded as progress, the native D cannot be translated forward to its progress formula in the first interpretation. Matrix congruence with an atomic target also cannot be assumed: its usual proof uses precisely the atomic case introduction being removed.

This is a limitation of the proposed deletion with the displayed natural interpretations, not a theorem excluding every possible unrelated bi-interpretation of every redesign. A possible new design could treat atomic D certificates as auxiliary judgments only and restrict genuine object formulas to a padded language. Such a design must specify and charge its object-language validation, rather than silently counting all accepted raw formulas as object theorems.

10 Further research questions

1. **Can the comparator be shortened or bypassed?** The factor-map method is explicit and uses only equation induction, but its finite term expands substantially. A direct predecessor cycle that requires no Boolean comparison could reduce both the proof and the generated certificates.

2. **How general is finite cancellation as a logical bootstrap?** Here successor can be cancelled on Boolean terms before it can be cancelled on arbitrary numbers. That recovers restricted implication, then precisely the contextual cases needed for arithmetic. Determine which other weak numerical signatures admit this route to their full logical interface.

3. **How general is transient initialization?** If a required orbit admits a known successor-state preimage, one extra application of its update supplies that initial state. Identify an abstract class of finite reset mechanisms sufficient to recover arbitrary initialization in other restricted recursion bases.

4. **Can the direct SR checker beat all three coordinates at once?** The direct program beats the incumbent normalized-byte minimum. An auxiliary I program now beats all three old minima by one unit each, while other I profiles improve individual coordinates further. Achieving the same joint improvement for the direct SR syntax would connect that stronger implementation comparison to the new arithmetic grammar. All parsing, scope, image, freshness, and induction restrictions must remain charged and justified.

5. **Can the tied-seed obstruction extend beyond term translations?** The bounded-coordinate theorem excludes ordinary addition, selection, and pairing terms. It does not settle definable relational or multi-coordinate interpretations. A model-theoretic invariant stable under such interpretations would be a substantially stronger negative result.

6. **Can the whole proof be machine checked?** The highest-value targets are the factor-map identities, the guarded literal induction, fixed-parameter cycle identities, the native Boolean case compiler, and the internal interpretation composites. The supplied primitive certificates and numerical evaluations are useful checks but cannot replace that formalization.

7. **What should the simplicity benchmark fix?** Source language, runtime primitives, input grammar, malformed-input behavior, and the treatment of staged descriptions all affect an implementation comparison. The current metric is reproducible once those choices are fixed. It does not supply a universal lower bound on mathematical axiomatizations.

The closed result is a new native arithmetic and logical presentation, a complete written bi-interpretation, and a direct checker below the supplied byte frontier. Absolute simplestness across arbitrary languages, domains, and implementations remains an open optimization problem.

A The complete measured programs

The principal program directly checks the new SR computation schemas. Its complete source, including JSON input, all validation and logical rules, and the verdict protocol, is charged in the measurement.

```
from json import load
k = 0
i = lambda a, t=None, d=0: [i(b, t, d + b' \x02\x01\x04'[len(a)]) for b in a] if isinstance(a, list)
    else a if k <= a < d else d % 2 and i(t, d=1)

def c(a, g):
    global k
    if i(a) in g:
        return
    m, s, u, t = q()
    if m:
        c(m, g)
        a = i(t, s) if m in ([s, u], not s and [t]) and i(t, u) == a else a
        h = m[2:]
    elif t:
        h = a = i(a[2], t, 2)
    else:
        n, t, u = s
        h = [s, i(u, i([*n, t, u], d=1), 1) if n else [t]]
    k -= 1
    c(i(a, [k]), g + [i(h, k if m or len(a) % 2 == m else 0)])
try:
    q = load(open(0)).pop
    c(q(), [])
except:
    print('REJECT')
    raise
print('ACCEPT')
```

The auxiliary I byte profile differs only in its zero-counter computation result. Its Eq/Q conditional-expression refactoring preserves exactly the incumbent accepted language.

```
from json import load
k = 0
i = lambda a, t=None, d=0: [i(b, t, d + b' \x02\x01\x04'[len(a)]) for b in a] if isinstance(a, list)
    else a if k <= a < d else d % 2 and i(t, d=1)

def c(a, g):
    global k
    if i(a) in g:
        return
    m, s, u, t = q()
    if m:
        c(m, g)
        a = i(t, s) if m in ([s, u], not s and [t]) and i(t, u) == a else a
        h = m[2:]
    elif t:
        h = a = i(a[2], t, 2)
    else:
        n, t, u = s
        h = [s, i(u, i([*n, t, u], d=1), 1) if n else t]
    k -= 1
    c(i(a, [k]), g + [i(h, k if m or len(a) % 2 == m else 0)])
try:
    q = load(open(0)).pop
    c(q(), [])
except:
    print('REJECT')
    raise
print('ACCEPT')
```

The auxiliary I structural profile shortens the iterable opener. Its extra successful JSON aliases and their soundness are specified in Section 6.

```
from json import load
k = 0

def i(a, t=None, d=0):
    try:
        return [i(b, t, d + b' \x02\x01\x04'[len(a)]) for b in a]
    except:
        return a if k <= a < d else d % 2 and i(t, d=1)

def c(a, g):
    global k
    if i(a) in g:
        return
    m, s, u, t = q()
    if m:
        c(m, g)
        if m in ([s, u], not s and [t]) and i(t, u) == a:
            a = i(t, s)
        h = m[2:]
    elif t:
        h = a = i(a[2], t, 2)
    else:
        n, t, u = s
        h = [s, i(u, i([*n, t, u], d=1), 1) if n else t]
    k -= 1
    c(i(a, [k]), g + [i(h, k if m or len(a) % 2 == m else 0)])
try:
    q = load(open(0)).pop
    c(q(), [])
except:
    print('REJECT')
    raise
print('ACCEPT')
```

The optional initialized-I predecessor-packet profile is strictly below all three old coordinate minima in a single program. Its changed packet protocol, soundness, and canonical completeness are proved in Section 6. A readable equivalent is also supplied.

```
from json import *
k = 0

def i(a, t=0j, d=0):
    try:
        return [i(b, t, d + b' BAD'[len(a)] % 8) for b in a]
    except:
        return a if k <= a < d else d % 2 and i(t, d=1)

def c(a, g):
    global k
    if i(a) in g:
        return
    m, s, u, t = q()
    if m:
        c(m, g)
        a = m in ([s, u], not s and [t]) and i(t, u) == a and i(t, s) or a
        h = m[2:]
    elif t:
        h = a = i(a[2], t, 2)
    else:
        n, t, u = s
        h = [[n and [n], t, u], i(u, s, 1) if n else t]
    k -= 1
    c(i(a, [k]), g + [i(h, k if m or len(a) % 2 == m else 0)])
try:
    c((q := load(open(0)).pop)(), [])
except:
    print('REJECT')
    raise
print('ACCEPT')
```

B Reproduction and attribution

Run `python3 verify.py` from the archive directory to remeasure the complete programs, replay the certificates and soundness controls, and execute both arithmetic diagnostic suites. Run `python3 build.py` to compile this document. The reports record CPython's version because `ast.unparse` is part of the measurement convention. No submitted Python expression is evaluated by either theorem checker; input is JSON data. The diagnostic evaluators and source measurer are not imports of the small checkers.

`evidence/source_snapshot.json` identifies all nine supplied ZIPs by Git blob hash, SHA-256, and byte length. The complete scan of 261 Python source occurrences (164 distinct contents) is in `evidence/corpus_metrics.json`. The smaller non-checker files are classified in `evidence/nonchecker_classification.json`. The ten nested historical ZIP references were also scanned separately: 154 Python source occurrences, with no change to the current frontier. Their measurements are in `evidence/nested_reference_metrics.json`. Frozen incumbent byte/token and AST leaders and the unchanged measurement utility are included in `reference/`. The new checker profiles are derived from the generated-assumptions draft; its numerical and logical machinery is attributed, not claimed as new. The comparator's factor-map strategy is adapted from the scalar predecessor appendix; the transient seeds, shifted factor maps, reset-cycle predecessor, finite-Boolean native bootstrap, and tied-seed obstruction are the new mathematical developments in this package.

References

[1] VladimirReshetnikov/ProveIt, `docs/drafts`, commit `2fdcf5bf988bead821c416649f713f98b7ed8776`, retrieved 10 November 2026. https://github.com/VladimirReshetnikov/ProveIt/tree/2fdcf5bf988bead821c416649f713f98b7ed8776/docs/drafts.

[2] ProveIt draft `ipra_shared_continuation_2026-11-10.zip`, especially the section "A genuinely smaller numerical grammar: the precise unresolved step," and its scalar predecessor appendix. Snapshot [1].

[3] ProveIt draft `ipra_successor_recursion_2026-11-09.zip`, the two-coordinate successor recursor and direct PR compiler. Snapshot [1].

[4] ProveIt draft `ipra_generated_assumptions_2026-11-10.zip`, the complete native I-checkers, known-instance cut, source metric, logical reconstruction, equality reflection, and scalar PR compiler. Snapshot [1].

[5] ProveIt draft `ipra_atomic_transport_2026-11-10.zip`, atomic transport and its attributed logical identity expansions. Snapshot [1].

[6] S. Salehi, "Decidable formulas of intuitionistic primitive recursive arithmetic," *Reports on Mathematical Logic* 36 (2002), 55–61. https://rml.tcs.uj.edu.pl/rml-36/36-SALEHI.pdf.